

# SimCSE on an SNLI subset — final report (U2T02)

**Team:** Julio Cesar De Aquino Castellanos, Gustavo Alexander Fuentes Marin, Valeria Nicol Hernandez Leon, Ricardo Daniel Horta Sanchez, Jose Angel Pech Xool, Lorena Danae Perez Lopez.

We reproduce unsupervised and supervised SimCSE (Gao, Yao & Chen, EMNLP 2021) with `bert-base-uncased` trained on a 100k-record SNLI subset, evaluate both on STS-B, run one ablation per mode, explain the gap to the paper, and publish both models on the Hugging Face Hub. Sections 3–5 include the per-part reports from `reports/` verbatim (headings shifted one level); the source files remain the reference copies.

## Contents

1. [Part 1 — Objective and conceptual answers](#)
2. [Configuration](#)
3. [Benchmark and analysis](#)
4. [Ablations](#)
5. [Gap to the paper](#)
6. [Optional extensions \(Part 8\)](#)
7. [Published models](#)

## 1. Part 1 — Objective and conceptual answers

Source: `reports/ part1_ objective.md` (Part 1).

Paper references: Gao, Yao & Chen, *SimCSE*, EMNLP 2021 (arXiv 2104.08821). Every equation, table and quote below was checked against the arXiv PDF. In the paper, **Eq. 1** is the generic in-batch contrastive loss for a pair  $(x_i, x_i^+)$ . **Eq. 4** instantiates it for unsupervised SimCSE, with  $x_i^+ = x_i$  and two dropout masks  $z, z'$ . **Eq. 5** is the supervised loss with contradiction hard negatives. Our numbers come from `configs/*.json`, `data/ processed/ MANIFEST.json` and `reports/ benchmark_ table.md`.

### 1. Mechanics of Eq. 1 (and Eq. 5): how many negatives per step?

For anchor  $i$  in a mini-batch of  $N$  pairs, Eq. 1 is

$$\ell_i = -\log \left[ \frac{\exp(\text{sim}(h_i, h_i^+)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(h_i, h_j^+)/\tau)} \right], \text{sim} = \text{cosine}.$$

- **Numerator:** the exponentiated, temperature-scaled cosine between the anchor and its *own* positive. In unsupervised SimCSE (Eq. 4) the positive is the same sentence encoded a second time with an independent dropout mask.
- **Denominator:** a sum over the positives of **all  $N$**  examples in the batch. That includes the anchor's own positive ( $j = i$ ), so the loss is an  $N$ -way softmax classification: "which of the  $N$  candidates is my positive?" The other  $N - 1$  terms are **in-batch negatives**. They are the other

sentences' second views, not extra mined examples. In our code this is

```
cross_entropy(z1 @ z2.T /  $\tau$ , arange(N)) (src/train_simcse.py).
```

**Unsupervised, our batch N = 64** ( `configs/ unsupervised.json` ): each anchor is pushed away from **N – 1 = 63 negatives** in every step. This is the same batch as the paper's unsup BERT-base setting (Table A.1: 64).

**Supervised, Eq. 5.** Each example is a triplet (premise  $x_i$ , entailment  $x_i^+$ , contradiction  $x_i^-$ ), and the denominator sums over **both**  $\exp(\text{sim}(h_i, h_j^+)/\tau)$  and  $\exp(\text{sim}(h_i, h_j^-)/\tau)$  for  $j = 1..N$ . With full coverage an anchor would have (N – 1) entailment negatives plus N hard negatives, i.e.  $2N - 1$ . Our implementation adds every **non-null** contradiction in the batch as an extra column. Only **28.4%** of our supervised pairs have one (9,488 of 33,351, `sup_pairs_with_hard_negative_frac = 0.284`). With **our batch N = 128** ( `configs/supervised.json` ):

|                                                             | Entailment (in-batch) negatives | Hard negatives (contradictions)                                                      | Total per anchor       |
|-------------------------------------------------------------|---------------------------------|--------------------------------------------------------------------------------------|------------------------|
| Ours, <code>use_hard_negatives=true</code>                  | 127                             | $\approx 0.284 \times 128 \approx \mathbf{36}$<br>(expected value; varies per batch) | $\approx \mathbf{163}$ |
| Same batch, 100% coverage (hypothetical)                    | 127                             | 128                                                                                  | 255                    |
| Ours, Part 5 ablation <code>use_hard_negatives=false</code> | 127                             | 0                                                                                    | <b>127</b>             |
| Paper (N = 512, every premise has a contradiction)          | 511                             | 512                                                                                  | 1,023                  |

Every anchor sees the  $\sim 36$  contradictions present in its batch, but only 28.4% of anchors have **their own** contradiction, the one that is lexically closest to them and therefore the most informative hard negative. Turning this term on and off is exactly what the Part 5 supervised ablation ( `ablation_sup_no_hardneg` ) does. Its small effect ( $-0.23$  dev,  $-1.45$  test, see `reports/ablations.md` ) is consistent with this low coverage ( `reports/gap_analysis.md` ).

## 2. Temperature $\tau$

$\tau$  divides every cosine before the softmax. Cosines lie in  $[-1, 1]$ . With  $\tau = 0.05$  (our value and the paper's) they become logits in  $[-20, 20]$ , so small differences in cosine turn into large differences in probability.

- **Small  $\tau \rightarrow$  sharp softmax.** The loss is dominated by the negatives that are most similar to the anchor (the hardest ones), and their gradients get the largest weight. The objective strongly penalizes near-duplicate negatives and pushes them apart.

- **Large  $\tau \rightarrow$  flat softmax.** All negatives are weighted almost equally. The objective barely distinguishes a close negative from a far one, and the pressure to spread the embeddings out (uniformity) weakens.

Our own logs show how strong this effect is at  $\tau = 0.05$ . In the unsupervised baseline, positives have cosine  $\approx 0.83$  and random pairs  $\approx 0.28$ . The 0.55-point cosine gap becomes a logit gap of  $0.55 / 0.05 \approx 11$ . With 63 negatives, the loss is  $\approx 63 \cdot e^{-11} \approx 10^{-3}$ . That is why the training loss reached  $\sim 10^{-4}$  without any collapse ( `runs/run_log.jsonl` , notes of `unsup_seed42_bs64` ).

**What the paper found** (Appendix D, Table D.1, STS-B **dev** Spearman, cosine similarity with different  $\tau$ , plus a dot-product variant "N/A"):

| $\tau$    | N/A (dot product) | 0.001 | 0.01 | 0.05        | 0.1  | 1    |
|-----------|-------------------|-------|------|-------------|------|------|
| STS-B dev | 85.9              | 84.9  | 85.4 | <b>86.2</b> | 82.0 | 64.0 |

The effect is **not monotonic**. Performance peaks at  $\tau = 0.05$ . It falls sharply when  $\tau$  is too large (82.0 at 0.1 and 64.0 at 1, a 22-point loss). It falls only mildly when  $\tau$  is too small (84.9 at 0.001). The paper's conclusion: "with a carefully tuned temperature  $\tau = 0.05$ , cosine similarity is better than dot product". *Our inference, not stated in the paper:* the table does not name the model, but its  $\tau = 0.05$  value (86.2) matches the paper's final **supervised** BERT-base dev score (Table 7). The ablation was therefore most likely run in the supervised setting. The paper does not report a  $\tau$  sweep for unsupervised SimCSE.

### 3. SimCSE vs. Sentence-BERT (2019): why the objective matters for STS-B

SBERT-2019 ( `bert- base- nli- mean- tokens` , our reference row) trains a siamese BERT with a **classification objective** on NLI. The two sentence embeddings  $u, v$  are combined as  $(u, v, |u - v|)$ , and a softmax layer predicts entailment / neutral / contradiction (Reimers & Gurevych, 2019). The loss only requires that **some linear classifier on top of  $(u, v, |u - v|)$**  can separate the three labels. Nothing in it says that  $\cos(u, v)$  has to be high for paraphrases and low for unrelated sentences. Cosine geometry is, at best, a side effect. The classifier is thrown away at inference.

SimCSE's loss is **written in terms of the cosine** ( $\text{sim} = \text{cosine}$  in Eq. 1/4/5). It directly raises  $\cos(h_i, h_i^+)$  and lowers  $\cos(h_i, h_j)$  for every other sentence in the batch.

This difference matters for STS-B because of how we (and the paper) evaluate ( `src/evaluate.py` , Part 4). We embed each sentence, compute the **raw cosine** between the two embeddings, and report its Spearman correlation with the gold score. No regressor is trained on STS-B (the paper follows the same "no additional regressor" setting, §6.1). The evaluation reads exactly the quantity SimCSE optimizes, and that SBERT's objective only constrains indirectly. Two consequences show up in our numbers:

- With the same base model and NLI supervision, our supervised SimCSE beats SBERT-2019 (82.05 vs. 80.77 dev, 78.75 vs. 76.98 test). It does so while training on  $\sim 1/30$  of SBERT's NLI pairs (33k SNLI pairs vs.  $\sim 1\text{M}$  SNLI+MNLI).
- The in-batch negatives also add a term that SBERT does not have: pushing apart *all* unrelated sentences, not only the labeled pairs. This shows in uniformity,  $-3.36$  for our sup model vs.  $-3.05$  for SBERT-2019 (next section). Spearman is a *ranking* metric over all 1,379 test pairs, so spreading out unrelated sentences, and not only separating labeled pairs, is what makes the cosine ranking reliable.

## 4. Alignment and uniformity

Wang & Isola (ICML 2020) define two properties of normalized embeddings  $f(x)$ , both lower = better:

- **Alignment** =  $E \|f(x) - f(x^+)\|^2$  over positive pairs: positives should be close.
- **Uniformity** =  $\log E \exp(-2\|f(x) - f(y)\|^2)$  over random pairs: the embeddings should spread over the hypersphere.

**Why one loss does both.** Split  $-\log$  of the softmax in Eq. 1 into two terms:

$$\ell_i = -\text{sim}(h_i, h_i^+)/\tau + \log \sum_j \exp(\text{sim}(h_i, h_j^+)/\tau).$$

The first term only depends on the positive pair. Minimizing it raises the positive cosine, which on the unit sphere is the same as reducing  $\|h_i - h_i^+\|^2 (= 2 - 2\cos)$ . That is **alignment**. The second term is a log-sum-exp over the anchor's similarity to everything in the batch. Minimizing it pushes the anchor away from all other sentences, most of all the closest ones. This is a soft version of the uniformity potential. The paper makes this precise (§5, Eq. 6): as the number of negatives  $\rightarrow \infty$ , the loss tends to  $-(1/\tau) \cdot E[f(x)^T f(x^+)] + E_x \log E_{\{x^-\}} \exp(f(x)^T f(x^-)/\tau)$ . The first term keeps positives similar ("alignment"), and the second "pushes negative pairs apart" ("uniformity"). The paper also shows the second term flattens the singular-value spectrum of the embedding matrix, which reduces BERT's anisotropy (§5, Eq. 7).

**What our numbers show** (STS-B dev, `reports/ benchmark_ table.md` and `reports/ eval_ <run_ id>.json`; alignment uses the 264 dev pairs with gold  $\geq 4$ , uniformity uses the 2,910 unique dev sentences):

| Model                                     | Alignment ↓  | Uniformity ↓               | Dev   | Test  |
|-------------------------------------------|--------------|----------------------------|-------|-------|
| raw<br>bert-base-uncased<br>(mean)        | 0.195        | $-1.650$                   | 59.31 | 47.29 |
| Our unsup SimCSE<br>( unsup_seed42_bs64 ) | <b>0.305</b> | <b><math>-2.963</math></b> | 76.92 | 68.33 |
| Our sup SimCSE<br>( sup_seed42_bs128 )    | <b>0.220</b> | <b><math>-3.362</math></b> | 82.05 | 78.75 |

| Model                                    | Alignment ↓ | Uniformity ↓ | Dev   | Test  |
|------------------------------------------|-------------|--------------|-------|-------|
| SBERT-2019                               | 0.193       | −3.049       | 80.77 | 76.98 |
| Ablation: unsup,<br>same dropout<br>mask | 0.561       | −3.081       | 61.47 | 52.33 |
| Ablation: sup, no<br>hard negatives      | 0.190       | −3.086       | 81.82 | 77.30 |

- **Raw BERT has "good" alignment only because everything is close.** Its uniformity (−1.65) is by far the worst: the space is an anisotropic cone in which every pair, related or not, has a high cosine. This matches the paper's observation (§7: pre-trained embeddings "have good alignment, [but] their uniformity is poor").

- **Unsupervised SimCSE mainly buys uniformity.** Uniformity improves from −1.65 to −2.96, while alignment gets somewhat worse (0.195 → 0.305). Dropout views are a very weak positive signal, so the second term of the loss dominates. The net result is +17.6 dev points over raw BERT. The paper reports that unsup SimCSE improves uniformity "whereas keeping a good alignment". Ours gives up more alignment than that, which is consistent with training on a narrow SNLI-caption corpus (see `gap_analysis.md`).

- **Supervision brings alignment back.** Entailment pairs are real paraphrases, so the first term now carries semantic signal: alignment improves 0.305 → 0.220. The hard-negative and in-batch terms push uniformity further, to −3.36. This is the model that is best on both axes relative to the unsup one, and it is our best STS-B model, again as the paper reports (§7: supervised data "further amends alignment").

- **The two ablations separate the two terms.** With a *shared* dropout mask (`ablation_unsup_same_mask`), the positive is identical to the anchor, so the first term is constant and has zero gradient. Only the uniformity term is trained. The result is exactly that: uniformity of −3.08, *better* than the unsup baseline, but alignment collapses to 0.561, and STS-B drops to 61.47 dev / 52.33 test. Uniformity without alignment is not enough. Removing **hard negatives** (`ablation_sup_no_hardneg`) moves the balance the other way. Alignment is slightly better (0.190), but uniformity is clearly worse (−3.09 vs. −3.36), because the closest negatives (the contradictions) are gone from the second term. Test drops by 1.45.

## 2. Configuration

One row per training run, from `configs/*.json` (each run's `runs/ <run_id>/ config.json` is the same plus `library_versions` and `started`). Every ablation changes exactly one field of its baseline (last flag column). Shared by all four runs: base model `bert-base-uncased`, AdamW, weight decay 0, linear LR decay with no warmup, fp16 AMP, one Tesla T4 on Kaggle (base runs: torch 2.10.0 / Python 3.12; ablations: torch 2.11.0 / Python 3.13; transformers 5.18.0 for all). Gradient clipping: 1.0 for the unsupervised runs, none for the supervised ones. Dev = best periodic STS-B dev Spearman ×100 (the kept checkpoint); test was scored once for each run: by Part 6 for the two published models, then by Part 8 for the two ablations (after their

checkpoints were fixed by dev).

| Run                      | Role                     | Pooling (train → eval)                          | LR    | Batch | $\tau$ | Epochs | Dropout (hidden / attn) | Max len | Seed | Ablated flag             | Dev               | Test  |
|--------------------------|--------------------------|-------------------------------------------------|-------|-------|--------|--------|-------------------------|---------|------|--------------------------|-------------------|-------|
| unsup_seed42_bs64        | Unsupervised (published) | cls + MLP (train only) → eval cls before_pooler | 3e-05 | 64    | 0.05   | 1      | 0.1 / 0.1               | 32      | 42   | same_dropout_mask=false  | 76.92 (step 125)  | 68.33 |
| sup_seed42_bs128         | Supervised (published)   | cls → eval cls                                  | 5e-05 | 128   | 0.05   | 3      | 0.1 / 0.1               | 32      | 42   | use_hard_negatives=true  | 82.05 (step 350)  | 78.75 |
| ablation_unsup_same_mask | Ablation of unsup        | cls + MLP (train only) → eval cls before_pooler | 3e-05 | 64    | 0.05   | 1      | 0.1 / 0.1               | 32      | 42   | same_dropout_mask=true   | 61.47 (step 1000) | 52.33 |
| ablation_sup_no_hardneg  | Ablation of sup          | cls → eval cls                                  | 5e-05 | 128   | 0.05   | 3      | 0.1 / 0.1               | 32      | 42   | use_hard_negatives=false | 81.82 (step 150)  | 77.30 |

Training data (data/ processed/ MANIFEST.json): 165,529 unique SNLI sentences (unsupervised); 33,351 (premise, entailment) pairs, of which 9,488 (28.4%) have a contradiction hard negative (supervised). STS-B: 1,500 dev / 1,379 test pairs.

## 3. Benchmark and analysis

### 3.1 Benchmark table

Source: reports/ benchmark\_table.md (Part 6).

Spearman  $\rho \times 100$  between cosine similarity and gold scores on STS-B (sentence-transformers/ stsb; dev = 1,500 pairs, test = 1,379 pairs), protocol of src/evaluate.py (no STS-B fine-tuning). Generated by src/benchmark.py.

| Model                                         | Dev Spearman | Test Spearman | Alignment ↓ | Uniformity ↓ |
|-----------------------------------------------|--------------|---------------|-------------|--------------|
| Our unsupervised SimCSE ( unsup_seed42_bs64 ) | 76.92        | 68.33         | 0.305       | -2.963       |
| Our supervised SimCSE ( sup_seed42_bs128 )    | 82.05        | 78.75         | 0.220       | -3.362       |
| raw bert-base-uncased (mean pooling)          | 59.31        | 47.29         | 0.195       | -1.650       |
| SBERT-2019 ( bert- base- nli- mean- tokens )  | 80.77        | 76.98         | 0.193       | -3.049       |

| Model                          | Dev Spearman | Test Spearman | Alignment ↓ | Uniformity ↓ |
|--------------------------------|--------------|---------------|-------------|--------------|
| SimCSE paper (unsup, reported) | 82.50        | 76.85         | —           | —            |
| SimCSE paper (sup, reported)   | 86.20        | 84.25         | —           | —            |

### Gap to the paper (dev vs. dev, test vs. test)

| Mode                    | Ours dev | Paper dev | Δ dev | Ours test | Paper test | Δ test |
|-------------------------|----------|-----------|-------|-----------|------------|--------|
| Our unsupervised SimCSE | 76.92    | 82.50     | -5.58 | 68.33     | 76.85      | -8.52  |
| Our supervised SimCSE   | 82.05    | 86.20     | -4.15 | 78.75     | 84.25      | -5.50  |

### Notes

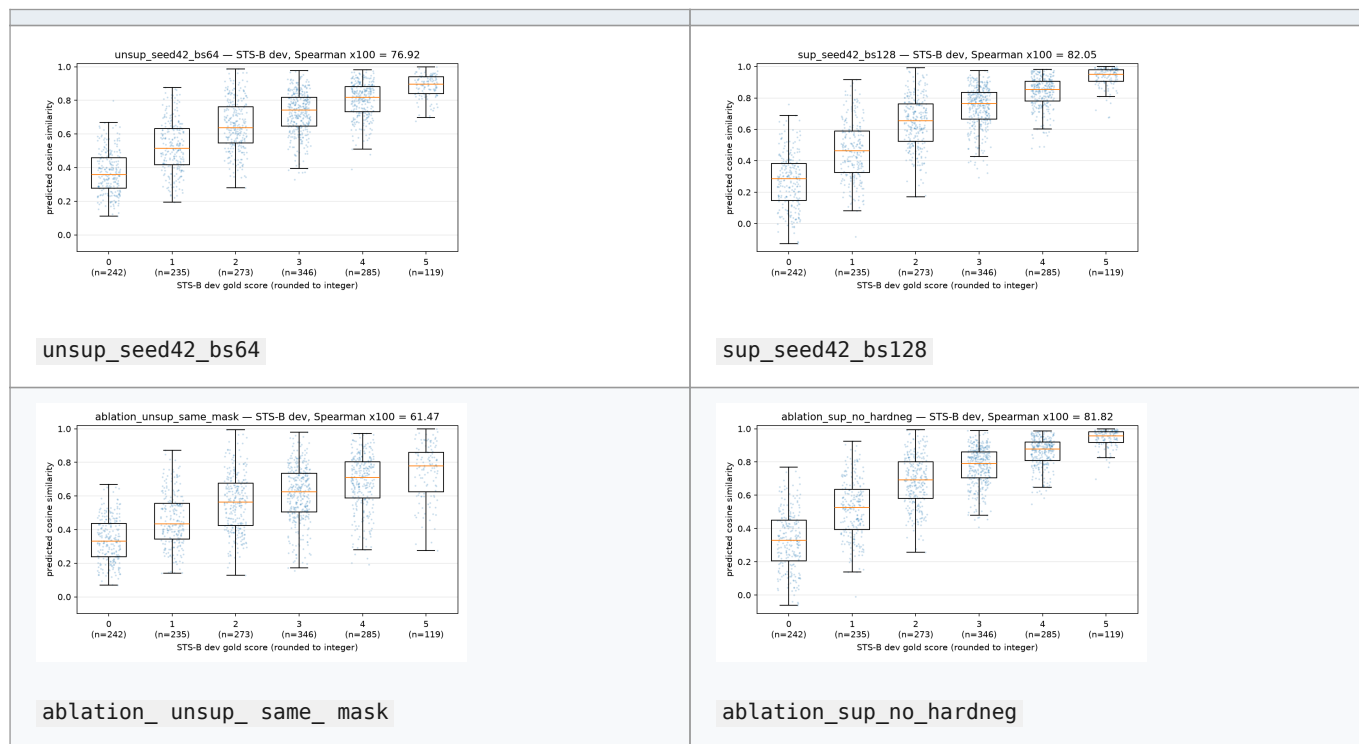
- **Dev** for our models is the best periodic dev evaluation during training, i.e. the score of the checkpoint that was kept (dev was used for checkpoint selection, so it is optimistically biased; test is not).
- **Test** for our models was scored exactly once, by `python src/ benchmark.py - - score- test`, on that kept checkpoint; raw BERT and SBERT-2019 test numbers come from the Part 4 Phase A run (`runs/run_log.jsonl`), not re-scored here.
- **Pooling**: unsup = [CLS] without the training MLP (`cls_before_pooler`); sup = [CLS] (trained without an MLP); raw BERT and SBERT-2019 = mean pooling.
- **Alignment / uniformity** (Wang & Isola, 2020; lower is better for both) are computed on STS-B **dev**, L2-normalized embeddings: alignment =  $\text{mean } \|f(x) - f(x^+)\|^2$  over the dev pairs with gold  $\geq 4/5$  ( $n = 264$ ); uniformity =  $\log \text{mean } \exp(-2\|f(x_i) - f(x_j)\|^2)$  over the 2,910 unique dev sentences. Trained models: `reports/ eval_ <run_id>.json` (Part 4 Phase B); references: `reports/ eval_ reference_ *.json` (this script). The paper rows have no comparable alignment/uniformity number (the paper's Fig. 2 uses a different positive set), hence —. Read the two together: raw BERT's low alignment comes with very poor uniformity (an anisotropic space where every pair is close), which is why it scores lowest on STS-B.
- **Paper rows** (Gao, Yao & Chen, EMNLP 2021, BERT-base): unsup dev 82.5 = Table 1, unsup test 76.85 = Table 5; sup dev 86.2 = Table 7, sup test 84.25 = Table 5. Never compare our dev with the paper's test (e.g. 76.92 dev vs. 76.85 test would wrongly suggest the unsupervised gap is zero).

## 3.2 Alignment and uniformity

The alignment / uniformity numbers are the last two columns of the table above (STS-B dev, Wang & Isola 2020; lower is better for both). No separate alignment-uniformity plot was produced in `reports/figures/`; per-run values are in `reports/ eval_ <run_id>.json`. Both trained models trade some alignment for much better uniformity than raw BERT ( $-2.96 / -3.36$  vs.  $-1.65$ ); the supervised model improves on both axes relative to the unsupervised one (alignment 0.220 vs. 0.305, uniformity  $-3.36$  vs.  $-2.96$ ), matching its higher STS-B score.

### 3.3 Similarity distributions

Cosine similarity of STS-B dev pairs against their gold score bucket, per run (Part 4 Phase B).



### 3.4 Retrievals (Part 4 Phase B)

Nearest neighbours over the 2,910 unique STS-B dev sentences. The full top-5 tables for six sample queries and three human-verified failure cases per run are in `reports/ retrievals_ <run_id>.md`; the summary and the failure analysis are reproduced here.

#### Unsupervised ( `unsup_seed42_bs64` )

##### Paraphrase retrieval over all dev positives

For each of the 264 dev pairs with gold  $\geq 4$ , rank of the gold paraphrase among all other dev sentences:

| Recall@1 | Recall@5 | MRR   | wrong top-1 rated unrelated (gold $\leq 1$ ) |
|----------|----------|-------|----------------------------------------------|
| 0.830    | 0.958    | 0.885 | 33                                           |

##### Failure discussion

**F3 ("The man is stirring the rice." → "The man is buttering the bread.", cosine 0.798, gold 0.4/5).** The correct neighbour, "A person is mixing a pot of rice.", is only rank 3 (0.750). The top-2 share the query's exact frame, "The man/cook is -ing the ", and its domain (cooking), but neither the action nor the object matches. Unsupervised SimCSE only learns to be invariant to dropout noise on the *same* sentence. Nothing in Eq. 1 says which tokens carry the meaning of an event, so sentences with the same syntactic template and lexical field stay close, as in pretrained BERT. Both supervised checkpoints retrieve "A person is mixing a pot of rice." as top-1 for this query (0.784 with hard negatives, 0.818 without). NLI entailment pairs explicitly align paraphrases that use different words ("stirring"  $\approx$  "mixing"), which is

exactly the signal missing here. The same weakness explains sample query Q4: "cracking eggs into a bowl" retrieves "mixing eggs in the bowl" (0.960), and the gold paraphrase "broke raw eggs" only ranks 6th.

**F1/F2 (earthquake headlines, cosine 0.844, gold 1.0/5).** Two headlines with an identical template ("quake strikes off ") describe different events. Annotators score event identity, but the model mostly encodes topic and form. Every checkpoint, supervised ones included, makes this mistake (see the other retrieval files). SNLI captions contain no news headlines and no examples where only a named entity or a number changes the meaning.

**Supervised** ( `sup_seed42_bs128` )

## Paraphrase retrieval over all dev positives

For each of the 264 dev pairs with gold  $\geq 4$ , rank of the gold paraphrase among all other dev sentences:

| Recall@1 | Recall@5 | MRR   | wrong top-1 rated unrelated (gold $\leq 1$ ) |
|----------|----------|-------|----------------------------------------------|
| 0.890    | 0.985    | 0.929 | 38                                           |

## Failure discussion

**F1/F2 (earthquake headlines, cosine 0.844, gold 1.0/5).** "6.4-magnitude quake strikes off Indonesia" and "6.9-magnitude quake strikes off Russia's Kuril Islands" are mutual top-1 neighbours. They share the template and topic, but annotators rated them nearly unrelated because they report different events. Supervision did not help here: the training data is SNLI image captions (no news headlines), and its contradictions are about scene content ("a man is sleeping" vs "a man is running"). None of them teaches that changing the place or the magnitude changes the meaning. The model therefore falls back on topic and form similarity. The remaining neighbours (other earthquake headlines at 0.69–0.83) show that this cluster is organised by topic, not by event.

**F3 ("Military plane crashes in south France" → "...southeastern Turkey, 1 dead", 0.780, gold 1.0/5)** is the same pattern: same event type and template, different instance.

**Contrast with the unsupervised model.** This checkpoint fixes the lexical-paraphrase failures of `unsup_seed42_bs64`. "The man is stirring the rice." now retrieves "A person is mixing a pot of rice." (0.784) instead of "...buttering the bread.", and in Q4 the gold paraphrase "broke raw eggs" moves from rank 6 to rank 1. What remains are "same topic, different specific event" errors. Within STS-B dev these occur mostly in the news domain, outside the SNLI training distribution. "Obama signs law aimed at barring Iran UN envoy" → "Obama urges no new sanctions on Iran" is also still a wrong top-1 here (0.769), but lower than without hard negatives (0.818, see `retrievals_ablation_sup_no_hardneg.md`).

## 4. Ablations

---

Source: `reports/ablations.md` (Part 5; test deltas added in Part 8).

One ablation per training mode. Each one changes **exactly one field** of the Part 3 baseline config. Seed (42), batch size, LR, epochs, temperature, pooling and the rest stay the same, so the delta comes from that one change.

`run_id` also differs, but it is only an identifier. All four runs trained on Kaggle (Tesla T4, fp16 AMP, 1 GPU used). Every score is Spearman  $\times 100$  on the STS-B **dev** split (1,500 pairs), computed by `src/evaluate.py`. "Best" means

the best of the periodic dev evaluations, which is also the checkpoint kept under `runs/ <run_id>/ checkpoint/`.

Test was left unscored for the ablations until Part 6 had scored the two baselines. Each ablation checkpoint was then scored on test exactly once, as a Part 8 extension; see [Test deltas](#) at the end.

## Results

| Pair         | Baseline run                   | Ablated run                           | What changed (one field)                                                                                                         | Dev baseline     | Dev ablated       | $\Delta$ dev |
|--------------|--------------------------------|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|------------------|-------------------|--------------|
| Unsupervised | <code>unsup_seed42_bs64</code> | <code>ablation_unsup_same_mask</code> | <code>same_dropout_mask: false → true</code> (both views reuse one forward pass and one dropout mask)                            | 76.92 (step 125) | 61.47 (step 1000) | <b>−15.4</b> |
| Supervised   | <code>sup_seed42_bs128</code>  | <code>ablation_sup_no_hardneg</code>  | <code>use_hard_negatives: true → false</code> (Eq. 5 → Eq. 1-style loss on (premise, entailment) pairs, only in-batch negatives) | 82.05 (step 350) | 81.82 (step 150)  | <b>−0.23</b> |

Reference: raw `bert-base-uncased` with mean pooling scores 59.31 dev.

Training curves (from each run's `results.json`):

| Run                                   | Dev range over all evals | Same-step $\Delta$ vs baseline (mean $\pm$ sd, n)                 | Training loss, last 250 steps |
|---------------------------------------|--------------------------|-------------------------------------------------------------------|-------------------------------|
| <code>unsup_seed42_bs64</code>        | 74.29 – 76.92            | —                                                                 | $\sim 1.3\text{e-}4$          |
| <code>ablation_unsup_same_mask</code> | 61.17 – 61.47            | $-13.8 \pm 0.6$ (n = 21, every step between $-12.9$ and $-15.6$ ) | $\sim 7.6\text{e-}7$          |
| <code>sup_seed42_bs128</code>         | 81.30 – 82.05            | —                                                                 | $\sim 0.46$                   |
| <code>ablation_sup_no_hardneg</code>  | 81.06 – 81.82            | $-0.36 \pm 0.17$ (n = 16, ablation lower at 16/16 steps)          | $\sim 0.40$                   |

## Noise estimate

Each config has only one seed (42). No baseline was rerun with another seed, so the noise level below is a reasoned estimate, not a measurement. It combines three sources:

1. **Sampling noise in the dev set.** On  $n = 1,500$  pairs at  $\rho \approx 0.82$ , the Fisher-z standard error of a correlation is  $(1 - \rho^2)/\sqrt{n - 3} \approx 0.0085$ , or **about 0.85-0.9 points** for Spearman. Both models are scored on the same pairs, so the paired SE of the difference is smaller. It is still likely a few tenths of a point. A paired bootstrap on the two checkpoints would measure it.
2. **Variation between checkpoints in one run.** The supervised baseline's dev score moves within a 0.75-point band (81.30-82.05) across its 16 evals. The ablation moves within 0.76 points. "Best dev" is the maximum of 16 noisy readings, so it is biased upward by an amount close to that band's spread.
3. **Variation between seeds.** We did not measure it. For BERT-base contrastive fine-tuning on STS-B it is usually a few tenths of a point in supervised mode and around 1 point in unsupervised mode, where SimCSE is known to be more sensitive to seed.

Taking these together, a reasonable noise floor for comparing two runs that each have one seed is **about  $\pm 0.5$  points (supervised) and about  $\pm 1$  point (unsupervised)**.

- **Unsupervised, -15.4.** This is more than 15 $\times$  the noise floor. The two dev curves never overlap: the ablation's best (61.47) is 12.8 points below the baseline's worst (74.29). The effect is real.
- **Supervised, -0.23.** This is **below** the single-seed noise floor. It is smaller than the sampling SE, smaller than either run's own variation between checkpoints, and smaller than typical seed variance. On its own it is **not a significant difference**. One observation points in the expected direction: the ablation is lower at all 16 matched eval steps (mean -0.36). Both runs share a seed and data order, so this comparison is partly paired. That consistency suggests a small real effect. It does not prove one, because the two runs' dropout RNG streams diverge after step 1: the ablated forward pass has fewer rows. Confirming it would take at least 2 more seeds per config, comparing means.

## Mechanisms

### Unsupervised: a shared dropout mask removes the alignment signal (real collapse)

Unsupervised SimCSE (Eq. 1) has only one source of augmentation: two dropout masks. In the baseline, each sentence appears twice in one forward pass, so rows  $i$  and  $B+i$  get independent masks and become two slightly different views. With `same_dropout_mask=true`, `src/train_simcse.py` runs one forward pass and uses  $z_1 = z_2 = z$ , so the two views are literally the same tensor. This has three consequences:

- After L2 normalisation the positive cosine is exactly 1. The positive logit is the constant  $1/\tau = 20$ , and its gradient is zero. The **alignment** term

of the objective (Wang & Isola) is gone.

- The only way left to lower the loss is to push the 63 in-batch negatives apart. The objective becomes **uniformity only**. The model can reach almost zero loss without learning anything about which sentences mean similar things.
- On the STS-B dev split, that matches what we see. The run gains a little early, from spreading out BERT's anisotropic [CLS] space; it lands slightly above raw-BERT mean pooling (59.31). Then it stops improving: dev stays at 61.2–61.5 for the whole epoch. The baseline, which does get an alignment gradient, reaches 76.92.

The paper's own ablation shows the same thing. In SimCSE §3, Table 3 ("Effects of different dropout probabilities"), "Fixed 0.1" means the same dropout mask for both views, and it causes a large drop on STS-B dev: 43.6, down from 82.5 for the paper's default (different dropout masks). The authors attribute it to representation collapse, because the positive pair no longer carries any information. Our drop (76.92 → 61.47) is smaller in absolute terms — likely because our setup (CLS pooling, `mlp_only_train`, smaller/easier SNLI-derived training set) doesn't collapse as completely in one epoch — but the direction and the mechanism are the same.

### The same log symptom had two different causes

Both unsupervised runs triggered the same automatic alert in `run_record.json`: *"final training loss < 1e-3: possible collapse (identical views?)"*. For the baseline it was a **false positive**. For this ablation it is a **real collapse**. The symptom (near-zero InfoNCE loss) is the same, but the cause differs:

|                       | unsup_seed42_bs64 <b>(baseline)</b>                                                                                                                                                                                        | ablation_ unsup_ same_ mask                                                                                             |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Views                 | independent dropout masks                                                                                                                                                                                                  | the same tensor ( <code>z1 = z2</code> )                                                                                |
| Positive cosine       | ~0.83 on average (min 0.78), measured locally                                                                                                                                                                              | exactly 1.0 by construction                                                                                             |
| Why the loss is small | $\tau = 0.05$ turns the gap between positive and negatives (0.83 vs ~0.28) into a logit gap of ~11, so $\text{loss} \approx 63 \cdot e^{-11} \approx 1e-3$ . The task is easy, but the positive term still has a gradient. | The positive logit is a constant 20. The loss falls only through negative repulsion, a task that is trivially solvable. |
| Loss plateau          | ~1e-4                                                                                                                                                                                                                      | ~7.6e-7 (~170× lower), reached by step 50                                                                               |
| Dev Spearman          | 76.92, well above raw BERT                                                                                                                                                                                                 | 61.47, flat for all 21 evals                                                                                            |
| Verdict               | not collapsed                                                                                                                                                                                                              | collapsed alignment                                                                                                     |

**Lesson:** a near-zero contrastive loss does not, on its own, diagnose collapse. With a small  $\tau$ , a model that has learned the task correctly can also drive the loss toward zero. Two checks tell the cases apart. First,

check whether the positive pair is trivially identical (positive cosine equal to 1, a single forward pass reused). Second, check whether dev Spearman moves during training. The alert threshold is a useful prompt to look closer, but it cannot decide the question.

## Supervised: hard negatives sharpen fine-grained separation, but the effect is small here

With `use_hard_negatives=true` (Eq. 5), each premise's softmax denominator also contains the contradiction hypotheses of the batch. SNLI contradictions usually overlap heavily with the premise in words and topic ("A man is playing a guitar" / "A man is not playing an instrument"), so they are negatives that sit close to the anchor. Pushing them away forces the encoder to encode meaning differences that go beyond topic and lexical overlap. That is what STS-B rewards in its middle range of similarity scores. Without hard negatives (Eq. 1 on the supervised pairs), the 127 in-batch negatives are mostly unrelated sentences. They are easy to separate, so the task gets easier. Its final loss is  $\sim 0.40$ , against  $\sim 0.46$  for the baseline, and it pushes less on the semantic boundary.

The measured effect ( $-0.23$  best,  $-0.36$  at matched steps) has the expected sign but is far smaller than in the paper. SimCSE Table 7 ("STS-B development results with different hard negative policies") reports  $+1.3$  points on STS-B dev from adding contradiction hard negatives (no hard negative:  $84.9 \rightarrow$  with weighted contradiction:  $86.2$ ; contradiction alone already reaches  $86.1$ , so most of the gain comes from adding the hard negative at all, not from the weighting). Likely reasons:

- **Hard-negative coverage, the main cause (see `reports/gap_analysis.md` for the full derivation).** The paper's 314k SNLI+MNLI triplets (Table 4) give a hard negative for every anchor. Our 100k-row SNLI subset gives 33,351 triplets, but only 9,488 of them (28.4%) have a contradiction hypothesis for the same premise — most anchors train with no hard negative at all. If the paper's  $+1.3$ -point gain (Table 7,  $84.9 \rightarrow 86.2$ ) scales linearly with coverage, we'd expect  $1.3 \times 0.284 \approx 0.37$  — almost exactly the  $+0.36$  we measured at matched steps. Coverage alone explains nearly all of the shortfall versus the paper's effect size.
- **Early best checkpoints.** Both runs peak early (steps 150 and 350 of 780), so most of the training where hard negatives would matter does not affect the selected checkpoint.
- **A large batch.** With batch size 128, the in-batch entailment negatives already supply much of the uniformity signal.

With one seed per config, we can only say that hard negatives help by at most a few tenths of a point in this setup. We cannot claim a significant gain.

## Test deltas (Part 8)

After Part 6, the two kept ablation checkpoints were scored on STS-B **test** (1,379 pairs) exactly once. We used `evaluate_sts` from `src/evaluate.py` with each run's own `eval_pooling`, seed 42 and batch 64, the same procedure as `benchmark.py - - score- test`. The results are in

`results.test_spearman` of their lines in `runs/run_log.jsonl`. The checkpoints had already been chosen by dev, so test played no part in any selection.

| Pair                                  | Test baseline | Test ablated | $\Delta$ test | $\Delta$ dev (for comparison) |
|---------------------------------------|---------------|--------------|---------------|-------------------------------|
| Unsupervised<br>( same_dropout_mask ) | 68.33         | 52.33        | <b>-16.00</b> | -15.4                         |
| Supervised<br>( use_hard_negatives )  | 78.75         | 77.30        | <b>-1.45</b>  | -0.23                         |

The unsupervised collapse carries over to test at the same size: the ablation falls to 5.0 points above raw BERT's 47.29. The supervised delta keeps its sign but is about 6× larger on test than on dev. That makes the "hard negatives help" reading more plausible, but it is still one seed and one reading. The test Fisher-z SE at  $\rho \approx 0.78$  is about 1 point per model (less for the paired difference), so -1.45 is suggestive, not significant. Do not compare it with the paper's +1.3, which is a **dev** number (Table 7).

## 5. Gap to the paper

Source: `reports/gap_analysis.md` (Part 6).

All numbers are STS-B Spearman ×100, BERT-base. We compare dev with dev and test with test (see `reports/ benchmark_ table.md`). Paper numbers: Gao, Yao & Chen (EMNLP 2021). Unsup dev is from Table 1 and unsup test from Table 5. Sup dev is from Table 7 and sup test from Table 5. Paper setup details below come from §3, §4, Table 4, Table 6 and Appendix A (Table A.1), checked against the arXiv PDF.

### The gap

| Mode                                  | Ours dev | Paper dev | $\Delta$ dev | Ours test | Paper test | $\Delta$ test | Our dev→test drop | Paper dev→test drop |
|---------------------------------------|----------|-----------|--------------|-----------|------------|---------------|-------------------|---------------------|
| Unsupervised<br>(unsup_ seed42_ bs64) | 76.92    | 82.5      | <b>-5.58</b> | 68.33     | 76.85      | <b>-8.52</b>  | 8.59              | 5.65                |
| Supervised<br>(sup_ seed42_ bs128)    | 82.05    | 86.2      | <b>-4.15</b> | 78.75     | 84.25      | <b>-5.50</b>  | 3.30              | 1.95                |

For comparison, the dev→test drop is 12.02 for raw `bert-base-uncased` (59.31 → 47.29) and 3.79 for SBERT-2019 (80.77 → 76.98). STS-B test is harder than dev for every model. It is hardest for models that stay close to raw BERT.

The test gap is larger than the dev gap in both modes: by 2.94 points for unsupervised and 1.35 for supervised. Comparing our dev 76.92 with the paper's test 76.85 would hide all of this, which is why the two splits are kept apart.

## Setup differences

|                                | Paper (BERT-base)                                                                   | Ours                                                                                                                                                                    | Source                                                  |
|--------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Unsup training data            | 10 <sup>6</sup> sentences sampled from <b>English Wikipedia</b>                     | <b>165,529</b> unique SNLI sentences (premises and hypotheses of a 100k-record subset)                                                                                  | §3 fn. 2, §4; <code>data/processed/MANIFEST.json</code> |
| Sup training data              | SNLI + MNLI, <b>314k</b> (premise, entailment, contradiction) triplets              | <b>33,351</b> SNLI (premise, entailment) pairs from the 100k-record subset (no-consensus records dropped, then sampled). Only <b>9,488 (28.4%)</b> have a contradiction | §4, Table 4; MANIFEST                                   |
| Hard negatives                 | One per triplet (every anchor)                                                      | One for 28.4% of anchors (first contradiction for the same premise); 71.6% have none                                                                                    | §4 fn. 7; MANIFEST <code>hard_negative_rule</code>      |
| Unsup bs / LR / epochs         | 64 / 3e-5 / 1                                                                       | 64 / 3e-5 / 1 (same)                                                                                                                                                    | Table A.1; <code>configs/unsupervised.json</code>       |
| Sup bs / LR / epochs           | <b>512</b> / 5e-5 / 3                                                               | <b>128</b> / 5e-5 / 3 (LR not re-tuned for the smaller batch)                                                                                                           | Table A.1; <code>configs/supervised.json</code>         |
| Optimizer steps (unsup)        | 10 <sup>6</sup> / 64 $\approx$ 15,600                                               | 2,586 (best checkpoint at <b>step 125</b> = 8,000 sentences seen)                                                                                                       | <code>runs/unsup_seed42_bs64/results.json</code>        |
| Optimizer steps (sup)          | 314k / 512 $\times$ 3 $\approx$ 1,840 (942k anchor-views)                           | 780 (100k anchor-views); best at step 350                                                                                                                               | <code>runs/sup_seed42_bs128/results.json</code>         |
| $\tau$ , max length, dropout   | 0.05, 32, 0.1                                                                       | same                                                                                                                                                                    | configs                                                 |
| Pooling                        | [CLS] + MLP. Unsup drops the MLP at test time; the paper's final sup model keeps it | Unsup: same. Sup: no MLP at all ( <code>mlp_only_train: false</code> builds none)                                                                                       | Table 6; <code>src/train_simcse.py</code>               |
| Grad clipping                  | HF default (1.0)                                                                    | unsup 1.0; <b>sup none</b> ( <code>max_grad_norm: null</code> )                                                                                                         | configs                                                 |
| Dev eval frequency / selection | every 250 steps, keep best                                                          | every 125 (unsup) / 50 (sup) steps, keep best                                                                                                                           | App. A; configs                                         |

|                       | Paper (BERT-base)                                                                                                                                                                                        | Ours                                                                                                                                                                                                                           | Source                                  |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Hyperparameter search | grid of batch $\in \{64, 128, 256, 512\} \times \text{LR} \in \{1e-5, 3e-5, 5e-5\} = \mathbf{12 \text{ configs per model}}$ , plus ablations of $\tau$ , $p$ , pooling and hard-negative $\alpha$ on dev | <b>1 config per mode, no search.</b> The unsup config copies Table A.1. The sup config uses the paper's LR with a 4× smaller batch. The two Part 5 runs are ablations (one method flag each), not a search                     | App. A; <code>runs/run_log.jsonl</code> |
| Seeds                 | best-checkpoint numbers for one setting                                                                                                                                                                  | 1 seed (42) per config                                                                                                                                                                                                         | run log                                 |
| Hardware / precision  | —                                                                                                                                                                                                        | 1× Kaggle Tesla T4, fp16 AMP; 618 s (unsup), 434 s (sup)                                                                                                                                                                       | run log <code>hardware</code>           |
| Things that broke     | —                                                                                                                                                                                                        | Unsup: "loss < 1e-3, possible collapse" alert. Investigated and found to be a <b>false positive</b> (positive cosine $\approx 0.83$ , not 1; the low loss comes from $\tau = 0.05$ ). Sup: nothing ( <code>notes</code> empty) | run log <code>notes</code>              |

## Attribution of the gap

These are estimates. Every config has one seed, and we did not re-run with the paper's data. Where the paper has a controlled comparison, we extrapolate from it; otherwise the share is reasoned from our training curves. The single-seed noise floor estimated in `reports/ablations.md` is about  $\pm 0.5$  (sup) and  $\pm 1$  (unsup). A residual smaller than that cannot be told apart from zero.

**Supervised: −4.15 dev / −5.50 test**

| Factor                                                                   | Dev (pts)                               | Test (pts)  | How the number was obtained                                                                                                                                                                                                                                                                                                               |
|--------------------------------------------------------------------------|-----------------------------------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Training data size and source</b> (33k SNLI pairs vs. 314k SNLI+MNLI) | <b>~2.1</b> (1.5-3)                     | <b>~3.1</b> | Paper Table 4, entailment-only positives: 134k → 314k pairs (2.3×) gives 84.1 → 84.9, i.e. +0.8, or ~0.65 per doubling. We have 9.4× fewer pairs (3.2 doublings), so ~2.1 dev. On test we add ~1.0 of the extra dev→test drop (3.30 vs. 1.95): a single-genre SNLI caption corpus transfers worse to the test genres than SNLI+MNLI does. |
| <b>Hard-negative coverage</b> (28.4% of anchors vs. 100%)                | <b>~1.0</b>                             | <b>~1.0</b> | The paper gains +1.3 from hard negatives (Table 7, 84.9 → 86.2). We measured +0.23 (best) / +0.36 (matched steps). The missing ~1.0 is the gain we did not get. The next section explains why.                                                                                                                                            |
| <b>No hyperparameter search; sup batch 128 with an LR tuned for 512</b>  | <b>~0.3</b> (0-0.5)                     | <b>~0.3</b> | The paper reports that SimCSE is "not sensitive to batch sizes as long as tuning the learning rates accordingly". We did not tune the LR, so we assume a small loss. We tried 1 config against the paper's 12.                                                                                                                            |
| <b>Dev-selection optimism</b> (best of 16 dev evals)                     | (inflates our dev; shrinks the dev gap) | <b>~0.4</b> | Our sup dev curve spans 81.30-82.05. The selected 82.05 sits at the top of that band, so part of it does not carry over to test.                                                                                                                                                                                                          |
| Pooling without MLP, no grad clipping, fp16, eval frequency              | <b>~0</b>                               | <b>~0</b>   | Table 6: sup [CLS] w/o MLP = 86.2 = w/ MLP. Clipping, fp16 and a denser eval schedule have no documented effect of this size.                                                                                                                                                                                                             |
| <b>Unexplained / seed noise</b>                                          | <b>~0.75</b>                            | <b>~0.7</b> | Residual; comparable to the ±0.5 single-seed floor.                                                                                                                                                                                                                                                                                       |

| Factor       | Dev (pts)   | Test (pts)  | How the number was obtained |
|--------------|-------------|-------------|-----------------------------|
| <b>Total</b> | <b>4.15</b> | <b>5.50</b> |                             |

Roughly: on dev, data  $\approx$  50%, hard negatives  $\approx$  25%, hyperparameters  $\approx$  7%, unexplained  $\approx$  18%. On test, data  $\approx$  56%, hard negatives  $\approx$  18%, hyperparameters  $\approx$  5%, selection  $\approx$  7%, unexplained  $\approx$  13%.

Data size and hard-negative coverage are not independent. Both come from using one 100k-record SNLI subset, so the whole  $\sim$ 3.1 dev points ( $\sim$ 4.1 test) traces back to Part 2's data preparation, not to the training code.

### Unsupervised: **−5.58 dev / −8.52 test**

| Factor                                                                                          | Dev (pts)                           | Test (pts)                  | How the number was obtained                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------|-------------------------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Training data source and size</b> (165k short SNLI sentences vs. $10^6$ Wikipedia sentences) | <b><math>\sim</math>4.5</b> (3.5–5) | <b><math>\sim</math>6.5</b> | Our hyperparameters match Table A.1, so most of the gap has to come from the data. Our curve shows that data is the binding constraint: dev peaks at the <b>first</b> eval (step 125, after only 8,000 sentences). It then drifts <b>down</b> to 74.6 over the remaining 2,461 steps. More SNLI training hurts rather than helps, so the problem is not too few steps but the narrow caption-style domain. On test, the checkpoint (8k sentences into training) is still close to raw BERT, whose dev $\rightarrow$ test drop is 12.0. This accounts for $\sim$ 2 of the 2.94 extra dev $\rightarrow$ test drop. |
| <b>Compute / steps</b> (2,586 vs. $\sim$ 15,600)                                                | <b><math>\sim</math>0</b> directly  | <b><math>\sim</math>0</b>   | Not binding. The best checkpoint is at step 125 and later steps lower dev. More steps on the same data would not close the gap; more steps on Wikipedia-scale data would, but that effect is counted under data.                                                                                                                                                                                                                                                                                                                                                                                                 |

| Factor                                                                            | Dev (pts)                               | Test (pts)   | How the number was obtained                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------|-----------------------------------------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>No hyperparameter search</b>                                                   | ~ <b>0.5</b> (0-1)                      | ~ <b>0.5</b> | We copied the paper's grid winner (bs 64, LR 3e-5) instead of re-tuning it for a different, much smaller corpus. The early peak suggests a lower LR or a shorter schedule might have helped, but we tried 1 config against the paper's 12. |
| <b>Dev-selection optimism</b> (best of 21 evals; the best is 1.06 above the next) | (inflates our dev; shrinks the dev gap) | ~ <b>1.0</b> | The selected step-125 score (76.92) is an isolated maximum; every later eval falls in 74.3–75.9. A single high reading on 1,500 dev pairs does not carry fully to test.                                                                    |
| eval every 125 vs. 250 steps, fp16, pooling                                       | ~ <b>0</b>                              | ~ <b>0</b>   | Same pooling as the paper (MLP in training only, Table 6: 82.5). The denser eval schedule is, if anything, an advantage.                                                                                                                   |
| <b>Unexplained / seed noise</b>                                                   | ~ <b>0.6</b>                            | ~ <b>0.5</b> | Within the $\pm 1$ single-seed floor for unsup SimCSE.                                                                                                                                                                                     |
| <b>Total</b>                                                                      | <b>5.58</b>                             | <b>8.52</b>  |                                                                                                                                                                                                                                            |

Roughly: on dev, data  $\approx 80\%$ , hyperparameters  $\approx 9\%$ , unexplained  $\approx 11\%$ . On test, data  $\approx 76\%$ , selection  $\approx 12\%$ , hyperparameters  $\approx 6\%$ , unexplained  $\approx 6\%$ .

The unsupervised data shift is a change of **source**, not only of size. The paper never trains unsup SimCSE on NLI sentences. Note that the Part 6 handoff calls the paper's unsup corpus "NLI-derived", which is wrong (§3 fn. 2: "We randomly sample  $10^6$  sentences from English Wikipedia"). Because the paper has no Wikipedia-subsample ablation, the  $\sim 4.5$  is the residual left after the other factors, not a direct extrapolation. Its range is wider than the supervised data estimate.

## Part 5 finding: why our hard-negative effect is $\sim 5\times$ smaller than the paper's

The paper gets +1.3 dev from contradiction hard negatives (Table 7: 84.9 with none, 86.2 with  $\alpha = 1$ ; Table 4 shows the same 84.9  $\rightarrow$  86.2). Our Part 5 ablation ( `ablation_sup_no_hardneg` vs. `sup_seed42_bs128` ) gets +0.23 at the best checkpoint and +0.36 averaged over 16 matched eval steps. That is

**3.6-5.7× smaller**, about 5×.

The main cause is **coverage**, which `reports/ablations.md` does not list.

In the paper, every anchor has a contradiction hypothesis (§4: "for each premise and its entailment hypothesis, there is an accompanying contradiction hypothesis"). In our data only 9,488 of 33,351 anchors (28.4%) have one, because the 100k-record subset breaks up most premise triplets.

`src/train_simcse.py` adds only the non-null contradictions to the denominator. A batch of 128 therefore has ~36 hard negatives, against 512 in a paper batch of 512, and 72% of anchors never see a hard negative of their own. In absolute terms we train on ~33× fewer hard negatives (9.5k vs. 314k).

If the paper's gain scaled linearly with coverage, we would expect  $1.3 \times 0.284 \approx \mathbf{0.37}$ . We measured **0.36** (matched-step mean). Coverage alone accounts for nearly the whole 5× shortfall. The other reasons in `ablations.md` (less data overall, early best checkpoints, the large in-batch negative pool) may add to it, but they are not needed to explain the magnitude. Small fixes for `ablations.md`: the paper's supervised set is 314k triplets, not ~275k, and the coverage effect should be listed first.

A direct fix would be to rebuild `sup_pairs.jsonl` from the full SNLI (or SNLI+MNLI) so that every anchor has a contradiction, then re-run the ablation pair with  $\geq 3$  seeds. We expect that to recover most of the ~1.0-point hard-negative share of the supervised gap.

## Bottom line

- **Supervised (−4.15 dev / −5.50 test):** about three quarters of the gap is explained by the data subset: 9.4× fewer pairs (~2.1 / ~3.1) and 28% hard-negative coverage (~1.0). The rest is untuned hyperparameters (~0.3), dev-selection optimism on test (~0.4) and noise (~0.7).
- **Unsupervised (−5.58 dev / −8.52 test):** about four fifths of the gap is the training corpus (SNLI captions instead of  $10^6$  Wikipedia sentences). Dev peaking after 8k sentences and then declining shows that more training on this corpus would not help. On test, an early, dev-optimistic checkpoint adds ~1.0 more.
- Neither gap points to a bug in the objective or the evaluation. The eval harness reproduces raw BERT and SBERT-2019 to  $\pm 0.00$  (Part 4 Phase A). Our unsup model is 17.6 dev points above raw BERT. Our sup model beats SBERT-2019 by 1.3 dev and 1.8 test while training on 33k SNLI pairs against SBERT's ~1M SNLI+MNLI pairs (~1/30).

## 6. Optional extensions (Part 8)

---

One extension, limited on purpose (no new training, no GPU):

- **What:** STS-B **test** scored exactly once for the two Part 5 ablation checkpoints (`ablation_unsup_same_mask`, `ablation_sup_no_hardneg`). Part 5 deliberately left their `test_spearman` as `null` until Part 6 had scored the baselines. No model was trained, and no checkpoint or hyperparameter was chosen using test.

- **Data / tools:** the same STS-B test split as everywhere else ( `data/stsb/test.jsonl` , 1,379 pairs, from `sentence-transformers/ stsb` ). The existing checkpoints under `runs/ <run_id>/ checkpoint/` , scored with `evaluate_sts` from `src/evaluate.py` , using each run's `eval_pooling` , seed 42 and batch 64, the same procedure as `src/ benchmark.py - - score- test` . Run on CPU locally. The results were written into `results.test_spearman` (plus `test_scored_at` ) of those two existing lines in `runs/run_log.jsonl` . No new lines were added.
- **Results:** unsupervised same-mask ablation **52.33** test ( $\Delta$  **-16.00** vs. 68.33); supervised no-hard-negative ablation **77.30** test ( $\Delta$  **-1.45** vs. 78.75). The unsupervised collapse is confirmed on test at the same size as on dev (-15.4). The supervised effect has the same sign as on dev but is larger (-1.45 vs. -0.23). With one seed, that is suggestive, not significant. See §4, "Test deltas (Part 8)".

Nothing else was attempted under Part 8: no extra data, no other evaluation sets, no extra seeds or sweeps.

## 7. Published models

Both models are public on the Hugging Face Hub as `sentence-transformers` models (Transformer → Pooling(cls) → Normalize, cosine similarity), exported by `src/publish.py` . Each was verified by reloading it from the Hub into an empty cache and re-running `evaluate_sts` on `data/stsb/test.jsonl`: the reloaded test Spearman equals the recorded one to floating-point precision. The Hub repo id is recorded as `hub_repo_id` on each run's line in `runs/run_log.jsonl` .

| Hub repo                                                | Run                            | STS-B test (×100) | Model card                                                              |
|---------------------------------------------------------|--------------------------------|-------------------|-------------------------------------------------------------------------|
| <a href="#">pyrawn/ simcse- bert- base- snli- unsup</a> | <code>unsup_seed42_bs64</code> | 68.33             | <a href="#">model_cards/ simcse- bert- base- snli- unsup/ README.md</a> |
| <a href="#">pyrawn/ simcse- bert- base- snli- sup</a>   | <code>sup_seed42_bs128</code>  | 78.75             | <a href="#">model_cards/ simcse- bert- base- snli- sup/ README.md</a>   |